

Simple Jenkins Pipeline for CI/CD

Overview

This project demonstrates a basic Jenkins CI/CD pipeline that builds a Docker image, runs automated tests, and deploys the application as a Docker container.

Tools

Jenkins

Docker

GitHub

Jenkins Declarative Pipeline

Repository Structure

text

```
.
├── Jenkinsfile
├── Dockerfile
├── run-tests.sh
├── app/
└── README.md
```

Pipeline Stages

Build - Builds a Docker image tagged with the Jenkins build number.

Test - Runs the application test script inside a temporary Docker container.

Deploy - Stops any previous container and starts the newly built image on port 8080.

Post actions - Reports success or failure and cleans the Jenkins workspace.

Jenkinsfile

groovy

```
pipeline {
    agent any

    triggers { githubPush() }

    environment {
        IMAGE_NAME = 'simple-app'
        CONTAINER_NAME = 'simple-app-container'
    }

    stages {
        stage('Build') {
            steps { sh 'docker build -t ${IMAGE_NAME}:${BUILD_NUMBER} .' }
        }
        stage('Test') {
            steps { sh 'docker run --rm ${IMAGE_NAME}:${BUILD_NUMBER} ./run-
tests.sh' }
        }
        stage('Deploy') {
            steps {
                sh '''
                    docker rm -f ${CONTAINER_NAME} || true
                    docker run -d --name ${CONTAINER_NAME} -p 8080:8080 $
${IMAGE_NAME}:${BUILD_NUMBER}
                '''
            }
        }
    }

    post {
```

```
        success { echo 'Build, test, and deployment completed successfully.' }
        failure { echo 'Pipeline failed. Check the stage logs for details.' }
        always { cleanWs() }
    }
}
```

Jenkins Configuration

Create a Jenkins Pipeline job.

Select Pipeline script from SCM and choose Git.

Enter the GitHub repository URL and set the script path to Jenkinsfile.

Configure a GitHub webhook or SCM polling trigger.

Ensure the Jenkins agent can execute Docker commands.

Push a commit and verify the build in the Jenkins dashboard.

Local Validation

bash

```
docker build -t simple-app:local .
```

```
docker run --rm simple-app:local ./run-tests.sh
```

```
docker run -d --name simple-app-container -p 8080:8080 simple-app:local
```

Expected Result

A successful Jenkins run should show green Build, Test, and Deploy stages. The deployed container should be accessible on port 8080 when the application exposes an HTTP service.